Explainability-Guided Distillation of Robust UAV Navigation Policies in a Noisy PyBullet Environment

Anonymous Submission

Affiliation withheld for review

Abstract. We study whether a robust UAV navigation policy can also be recovered as a compact analytical controller. To do so, we build a custom Gymnasium-compatible environment on top of PyBullet and evaluate all reported models directly in the hardest stage-4 setting, which combines trajectory-aware rewards, noisy observations, Kalman-filtered state estimation, aerodynamic drag, wind gusts, and drifting obstacles. These perturbations are chosen to target specific failure modes rather than to act as generic randomization. We train SAC, TD3, DDPG, and PPO in this single-shot stage-4 environment and then analyze the learned controllers with decision-tree surrogates, LIME, SHAP, action-vector fields, Q-value landscapes, and rollout-aware equation fitting. The strongest result is that policy quality and policy distillability separate clearly: the final SHAP-guided SAC equation preserves 4/4 held-out stage-4 successes, whereas DDPG preserves 2/4 and TD3 preserves 0/4 under the same evaluation seeds. The recovered SAC law has the structure of a damped contextual potential field even though no such controller was hard-coded into the reward, and it avoids the local-minimum failures that trap an untuned artificial potential field on the same map. We therefore frame the paper as an empirical study of explainability-guided policy distillation for noisy continuous UAV navigation, rather than as a new RL method.

Keywords: UAV navigation · reinforcement learning · Kalman filtering · explainable AI · policy distillation

1 Introduction

Learning-based UAV navigation is attractive because hand-crafted obstacle-avoidance rules often become brittle once sensing becomes noisy, obstacles drift, or dynamics deviate from the nominal model. At the same time, high-performing deep RL policies are difficult to audit, difficult to transfer, and difficult to compare beyond scalar reward. This raises a broader question: if a controller succeeds in a noisy environment, can its behavior be explained as a compact control law rather than only as a black-box network?

We address that question in a controlled simulation study. The environment is a custom Gymnasium-style task [3] backed by PyBullet [4], with a 2D quadrotor-like agent navigating through mixed-shape obstacles toward a fixed goal. All

experiments reported in the paper train directly in a single-shot stage-4 setting. That stage combines trajectory penalties, measurement noise, a Kalman filter [6], aerodynamic drag, wind gusts, and obstacle drift. These perturbations are not arbitrary. They target the main failure modes seen in early policies: poor state estimation, overconfident straight-line motion, and sensitivity to mild distribution shift. In that sense, the environment behaves like a structured robustness benchmark inspired by sim-to-real randomization work [16,13], but with perturbations that remain interpretable.

The second part of the paper focuses on explanation and distillation. We combine decision-tree surrogates, LIME [14], SHAP [10], vector-field plots, Q-value heatmaps, and rollout-aware equation extraction. The main empirical finding is that the best SAC controller behaves like a damped contextual potential field [8]: action depends primarily on raw goal displacement, velocity damping, and a short-range obstacle repulsion term. This structure was not hard-coded. It emerged from training and was recovered post hoc. By contrast, TD3 and DDPG can achieve strong task success while remaining much harder to summarize with the same interpretable controller family.

The contribution is therefore a combined environment-design, robustness, and explainability study with three concrete outcomes:

1. a staged noisy PyBullet environment for UAV navigation with a Kalman-filter state-estimation layer;
2. an explainability pipeline that goes beyond feature attribution and validates distilled equations in closed-loop rollouts;
3. empirical evidence that policy success and policy distillability are different properties, with SAC emerging as the most faithful analytical surrogate in our setting.

2 Related Work

Deep RL has been used widely for navigation and obstacle avoidance in both ground and aerial robots, including continuous-control drone settings [11,5]. However, much of that literature focuses on success rate and transfer robustness rather than on whether the learned policy can be turned back into a compact control law. Benchmarking work such as Arena-Bench [7] has also emphasized how sensitive obstacle-avoidance performance can be to environment dynamics and evaluation protocol. Our environment design follows that line of thought: we do not assume that clean-navigation success is enough to demonstrate robustness.

There is also a clear line of work on interpretable or explainable RL. Surveys by Milani et al. [12] and related explainable-robotics work [9] show that explanation methods in RL range from saliency and attribution to symbolic surrogates, hierarchical skills, and policy extraction. SHAP [10] and LIME [14] have become standard post hoc explanation tools, while policy distillation [15] and verification-oriented extraction methods such as VIPER [1] aim to replace black-box policies with simpler surrogates. More recent work on concept-based

policy distillation [2] further argues that interpretable latent concepts can preserve useful control structure.

Our work differs from those directions in two ways. First, the target is a continuous UAV-navigation controller, not a discrete toy domain or a concept-level explanation alone. Second, we treat *closed-loop fidelity* as the key evaluation criterion for distillation. A surrogate with high one-step R^2 but poor rollout behavior is not considered faithful enough. This is the central methodological difference from many explanation pipelines, which stop at importance rankings or local linear explanations.

Finally, combining filtering and RL for navigation is itself not new. Marino and Guglieri [11], for example, combine an interacting multiple model Kalman filter with PPO for drone navigation. Our contribution is therefore narrower and, we believe, more defensible: in a custom PyBullet UAV environment, we combine a structured noise-and-dynamics benchmark, a Kalman-filter ablation, and rollout-validated analytical distillation. The novelty lies less in any single ingredient than in the way they are tied together to show a measurable gap between policy success and policy distillability.

3 Environment and Training Setup

3.1 Custom Gymnasium/PyBullet Environment

The environment is a 2D navigation task implemented with a Gymnasium interface [3] and simulated in PyBullet [4]. The UAV starts near one of four corners of a 10.000 m square workspace and must reach a central goal while avoiding eight mixed-shape obstacles (spheres, cylinders, and boxes). The action is a 2D velocity command in $[-1, 1]^2$ scaled by a maximum speed of 15.000 m/s. The observation contains UAV position and velocity, goal displacement and distance, plus obstacle positions, radii, and shape codes.

Episodes run at 240.000 Hz for up to 18.000 s. Success occurs when the UAV reaches within 0.650 m of the goal, or remains within 1.000 m for 120 consecutive steps, without colliding. This definition prevents the agent from grazing the goal once and then drifting away.

3.2 Stage-4 Robustness Setup

All experiments in this paper use a single-shot stage-4 configuration. The reward combines a goal bonus, dense progress shaping, directional alignment, obstacle repulsion, a clearance bonus, a smoothness penalty, a near-miss penalty, a stall penalty, and a small per-step time cost. The state estimate is deliberately corrupted before it reaches the policy: position is perturbed by Gaussian noise with standard deviation 0.050 m, velocity by 0.100 m/s, the goal vector by 0.050 m, and obstacle positions by 0.080 m.

We then add dynamics and geometry perturbations that make memorized straight-line behavior brittle. The UAV is subjected to aerodynamic drag with

coefficient 0.12 and wind gust noise with standard deviation 0.35. Obstacles drift around anchored nominal locations with per-step drift standard deviation 0.018, damping 0.965, and a maximum anchored offset of 0.300 m. These perturbations are not generic domain randomization for its own sake. They are chosen to create the specific observation and dynamics mismatch that motivates filtering, robustness, and explainability in the single-shot stage-4 task.

3.3 Kalman Filter

In the stage-4 setup, the policy no longer sees the exact UAV state. Instead, the environment corrupts position and velocity with Gaussian noise and then fuses them through a Kalman filter:

$$\hat{\mathbf{x}}_{t|t-1} = \mathbf{F}\hat{\mathbf{x}}_{t-1|t-1}, \tag{1}$$

$$\mathbf{P}_{t|t-1} = \mathbf{F}\mathbf{P}_{t-1|t-1}\mathbf{F}^{\top} + \mathbf{Q}, \tag{2}$$

$$\mathbf{K}_t = \mathbf{P}_{t|t-1}\left(\mathbf{P}_{t|t-1} + \mathbf{R}\right)^{-1}, \tag{3}$$

$$\hat{\mathbf{x}}_{t|t} = \hat{\mathbf{x}}_{t|t-1} + \mathbf{K}_t\left(\mathbf{z}_t - \hat{\mathbf{x}}_{t|t-1}\right), \tag{4}$$

where $\hat{\mathbf{x}} = [x, y, v_x, v_y]^{\top}$ and $\mathbf{F}$ is the constant-velocity transition matrix. In our implementation, the filter does not alter the physical state. It only improves the observation delivered to the policy. This distinction matters: the learned agent still has to act under noisy goals, drifting obstacles, drag, and wind, but it acts from a better state estimate.

3.4 Training Protocol

All policies discussed in the paper are trained directly in stage 4. SAC is represented by a single-shot stage-4 run with Kalman filtering enabled. TD3, DDPG, and PPO were tuned with Optuna and then trained sequentially for 5M steps each on the same stage-4 environment with 8 parallel environments. We also trained a matched stage-4 SAC run without the Kalman filter to isolate the effect of state estimation.

The explainability and distillation sections focus on the SAC controller that yielded the strongest and most stable analytical surrogate under stage-4 evaluation.

4 Robustness Results

4.1 Algorithm Comparison

Figure 1 compares the smoothed evaluation success rate and reward curves for directly trained stage-4 SAC, TD3, DDPG, and PPO runs, clipped to the first 1.050 M timesteps for readability. The single-shot stage-4 SAC run still reaches reliable success fastest and remains the strongest policy overall in this window.

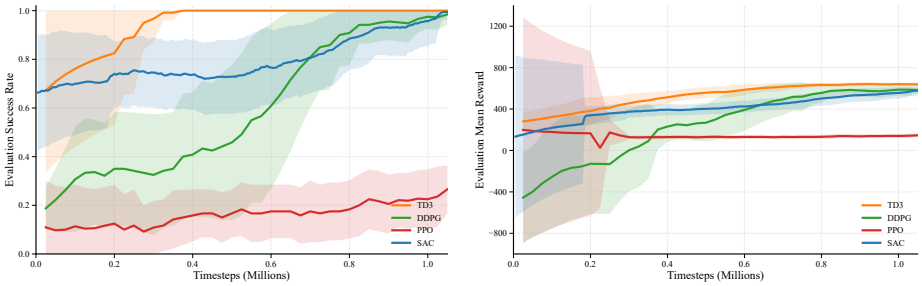

Fig. 1. Evaluation success rate (left) and mean reward (right) for directly trained stage-4 SAC, TD3, DDPG, and PPO runs. The plots are clipped to the first 1.050 M timesteps so that the early training dynamics can be compared on the same environment difficulty.

Table 1. Summary of best policy checkpoints and final SHAP-equation fidelity at stage 4.

Policy	Best eval succ.	Eq. succ.	Trace dev. (m)
SAC	1.000	1.000	2.154
TD3	1.000	0.000	7.075
DDPG	1.000	0.500	3.557
PPO	0.625	–	–

TD3 and DDPG both attain perfect evaluation checkpoints, but their curves are less stable and later regressions are visible outside the clipped window. PPO remains the weakest baseline in this environment, with a best recorded success rate of 62.5%.

Table 1 summarizes the strongest checkpoints and final distillation results. The important point is that high task success does not guarantee faithful distillation.

4.2 Kalman Filter Ablation

Figure 2 shows stage-4 single-shot SAC training with and without the Kalman filter, clipped to the first 1.050 M timesteps. The no-KF run can still reach successful checkpoints later in training, but it learns more slowly and is much more unstable. In our saved runs, the filtered single-shot SAC reached perfect evaluation success by 1.650 M steps, while the no-KF variant required 6.300 M steps to reach its best perfect checkpoint and later collapsed to zero success in its final evaluation.

The practical role of the filter is simple: it converts noisy position/velocity measurements into a smoother latent state before the policy maps observation to action. In our environment, that reduces spurious velocity corrections and makes the policy less brittle to the added drag, wind, and obstacle drift.

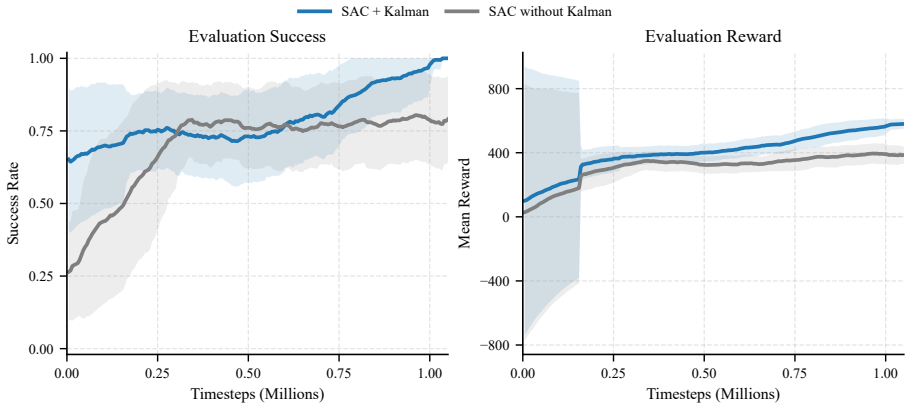

Fig. 2. Stage-4 single-shot SAC with and without the Kalman filter. The filter improves learning speed and stabilizes training by reducing the variance of the state estimate seen by the policy.

5 Explaining the Learned Policy

5.1 Potential-Field-Like Behavior

Figure 3 shows the core qualitative evidence that the main SAC policy learned a control strategy reminiscent of an artificial potential field [8] even though such a field was not explicitly coded into the reward. The action-vector field is plotted only over free space, so the arrows indicate how the controller bends around obstacles rather than cutting through them. The Q-value landscape forms a smooth basin around safe goal-reaching trajectories. Most importantly, the potential-field comparison now makes the failure mode explicit: an untuned APF creates a non-goal local minimum that traps the trajectory, whereas the trained stage-4 SAC policy escapes the same region and reaches the goal.

5.2 Decision Trees, LIME, and SHAP

We used three complementary post hoc explanation tools. A shallow decision-tree regressor provides discrete if/then rules; LIME gives local linear explanations around selected states; and SHAP supplies global feature-attribution summaries. For the main SAC policy, the decision-tree fit reached $R^2 = 0.826$ for the x-action and $R^2 = 0.670$ for the y-action. The learned rules split primarily on raw goal displacement for action x and on goal displacement together with repulsion for action y. That is already suggestive: the tree rediscovers a goal-seeking controller with directional obstacle corrections.

The SHAP and LIME panels in Fig. 4 make the same pattern more explicit. Goal displacement and the UAV state dominate attribution, while individual obstacle coordinates have lower global importance. However, a purely goal-directed

explanation is insufficient in closed loop. Obstacle-aware terms are needed to preserve safe behavior, especially in the final stage-4 evaluation. This is one reason why we do not stop at SHAP plots alone.

5.3 Rollout-Aware Equation Distillation

Based on these explanations, we fit a family of compact analytical controllers using raw goal displacement, velocity damping, and short-range obstacle repulsion terms. Importantly, we select the final equation by held-out rollout success first, then trace deviation, and only then by one-step R^2. This avoids the common failure mode where a surrogate looks excellent on sampled states but diverges quickly in closed loop.

The final SAC equation selected at stage 4 is

$$a_x = 0.0427 + 0.1442\,\texttt{goal_dx} - 0.0120\,v_x + 0.4725\,r_x, \tag{5}$$

$$a_y = -0.0624 + 0.1144\,\texttt{goal_dy} - 0.0071\,v_y + 0.6592\,r_y, \tag{6}$$

where (r_x, r_y) denotes a nearest-obstacle repulsion vector with exponential decay inside a 4.500 m threshold. On held-out validation rollouts, this equation preserved 4/4 successes and achieved a mean trajectory deviation of 2.154 m. The result is important because the same extraction pipeline does *not* work equally well for all successful policies: DDPG preserves only 2/4 successes and TD3 preserves 0/4 under the same stage-4 seeds.

6 Trajectory-Level Distillation Results

Figure 5 compares the original policies against their final stage-4 SHAP-guided equations. SAC is the clearest success case. Its analytical surrogate tracks the original policy closely enough to preserve all evaluated successes. DDPG is partially distillable but still inconsistent across seeds. TD3 has excellent one-step regression in some settings, yet its rollout behavior diverges badly; it is therefore an example of why closed-loop validation matters.

This gap between task success and distillability is the main empirical message of the paper. A policy can solve the task while still being hard to summarize as a compact control law. In our experiments, SAC appears to learn a smoother and more structured controller than TD3 or DDPG in the same noisy environment.

7 Discussion and Limitations

The most important lesson from these experiments is that robustness and interpretability are related but not identical objectives. Stage-4 SAC, TD3, and DDPG can all reach perfect evaluation checkpoints, yet only SAC consistently admits a compact analytical surrogate with acceptable closed-loop fidelity. In other words, solving the task is not enough; some policies remain far easier to compress into a control law than others.

That observation also clarifies the scope of the paper. We are not proposing a new RL algorithm, nor are we claiming the first use of Kalman filtering, SHAP, or LIME in navigation. The contribution is the synthesis: a structured noisy benchmark, a controlled state-estimation ablation, and a trajectory-level distillation pipeline that tests whether an interpretable surrogate can actually fly the task. Within that setup, SAC is the clearest positive case and TD3 is the clearest counterexample.

The work nevertheless has important limitations. The environment is custom and two-dimensional, so the conclusions are strongest as a case study rather than as a universal statement about all UAV controllers. The explanation pipeline is still post hoc: it reveals meaningful structure in the learned controller, but it does not guarantee causal sufficiency or formal interpretability. In addition, although the algorithm-comparison plots now use matched stage-4 single-shot runs, the deeper interpretability case study still centers on the SAC controller that produced the most faithful analytical surrogate.

Despite those limitations, the evidence supports one strong conclusion: if the goal is not only to learn a high-performing policy but also to recover a compact analytical surrogate, SAC is the most promising controller family among the methods tested here.

8 Conclusion

We introduced a custom Gymnasium/PyBullet UAV-navigation environment and evaluated all reported policies in a single-shot stage-4 setting that combines noisy observations, Kalman-filtered state estimation, drag, wind, and drifting obstacles. We then analyzed the learned policies with decision trees, LIME, SHAP, vector fields, Q-value heatmaps, and trajectory-level distillation.

The central finding is that SAC is not just the best-performing policy in our experiments; it is also the most *distillable*. Its final SHAP-guided equation preserves stage-4 rollout success and exposes a compact damped potential-field-like controller that was never hard-coded. This suggests a useful research direction for future work: evaluating learned policies not only by reward and success rate, but also by the fidelity of the simplest analytical controller that can reproduce them.

References

1. Bastani, O., Pu, Y., Solar-Lezama, A.: Verifiable reinforcement learning via policy extraction. In: Advances in Neural Information Processing Systems 31. pp. 2494–2504 (2018)
2. Bekkemoen, Y., Langseth, H.: Interpretable deep reinforcement learning via concept-based policy distillation. Machine Learning **114**, 288 (2025). https://doi.org/10.1007/s10994-025-06928-5
3. Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., Zaremba, W.: Openai gym (2016). https://doi.org/10.48550/arXiv.1606.01540

4. Coumans, E., Bai, Y.: Pybullet, a python module for physics simulation for games, robotics and machine learning. http://pybullet.org (2021), software
5. Huang, Z., Yang, Z., Krupani, R., Şenbaşlar, B., Batra, S., Sukhatme, G.S.: Collision avoidance and navigation for a quadrotor swarm using end-to-end deep reinforcement learning (2024). https://doi.org/10.48550/arXiv.2309.13285, accepted to ICRA 2024
6. Kalman, R.E.: A new approach to linear filtering and prediction problems. Journal of Basic Engineering **82**(1), 35–45 (1960). https://doi.org/10.1115/1.3662552
7. Kästner, L., Bhuiyan, T., Le, T.A., Treis, E., Cox, J., Meinardus, B., Kmiecik, J., Carstens, R., Pichel, D., Fatloun, B., Khorsandi, N., Lambrecht, J.: Arena-bench: A benchmarking suite for obstacle avoidance approaches in highly dynamic environments (2022). https://doi.org/10.48550/arXiv.2206.05728
8. Khatib, O.: Real-time obstacle avoidance for manipulators and mobile robots. The International Journal of Robotics Research **5**(1), 90–98 (1986). https://doi.org/10.1177/027836498600500106
9. Lee, K., Kim, S., Choi, J.: Adaptive and explainable deployment of navigation skills via hierarchical deep reinforcement learning (2023). https://doi.org/10.48550/arXiv.2305.19746
10. Lundberg, S.M., Lee, S.I.: A unified approach to interpreting model predictions. In: Advances in Neural Information Processing Systems 30. pp. 4765–4774 (2017)
11. Marino, F., Guglieri, G.: Beyond static obstacles: Integrating kalman filter with reinforcement learning for drone navigation. Aerospace **11**(5), 395 (2024). https://doi.org/10.3390/aerospace11050395
12. Milani, S., Topin, N., Veloso, M., Fang, F.: A survey of explainable reinforcement learning. arXiv preprint arXiv:2202.08434 (2022). https://doi.org/10.48550/arXiv.2202.08434
13. Peng, X.B., Andrychowicz, M., Zaremba, W., Abbeel, P.: Sim-to-real transfer of robotic control with dynamics randomization. In: 2018 IEEE International Conference on Robotics and Automation. pp. 3803–3810 (2018). https://doi.org/10.1109/ICRA.2018.8460528
14. Ribeiro, M.T., Singh, S., Guestrin, C.: "why should i trust you?": Explaining the predictions of any classifier. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 1135–1144 (2016). https://doi.org/10.1145/2939672.2939778
15. Rusu, A.A., Colmenarejo, S.G., Gulcehre, C., Desjardins, G., Kirkpatrick, J., Pascanu, R., Mnih, V., Kavukcuoglu, K., Hadsell, R.: Policy distillation (2015). https://doi.org/10.48550/arXiv.1511.06295
16. Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., Abbeel, P.: Domain randomization for transferring deep neural networks from simulation to the real world. In: 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems. pp. 23–30 (2017). https://doi.org/10.1109/IROS.2017.8202133

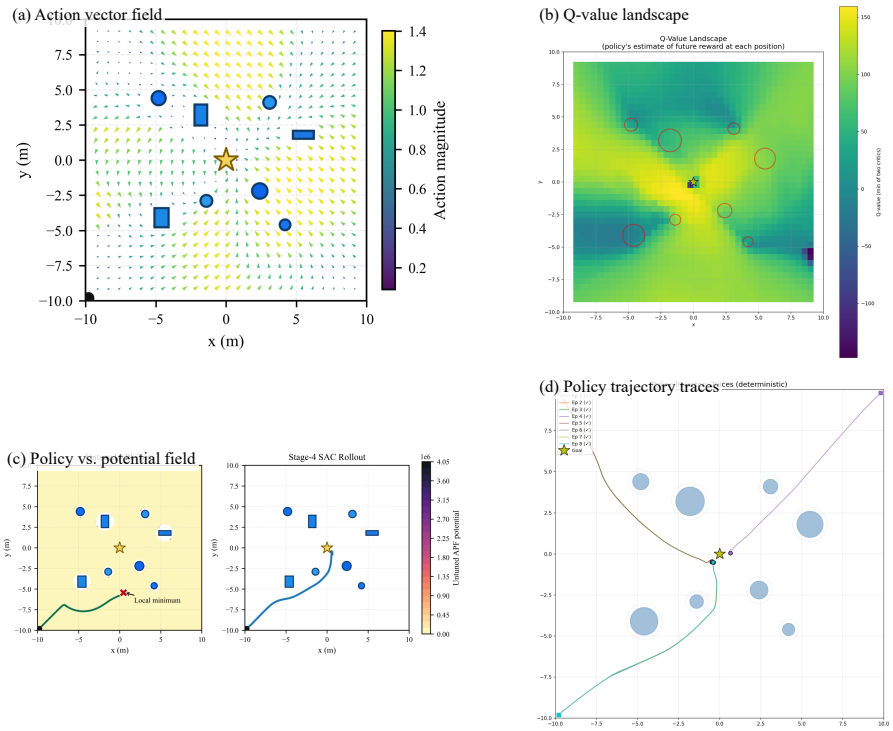

Fig. 3. Qualitative interpretation of the main SAC policy. (a) Action-vector field induced by the learned actor, plotted only in free space so obstacle interiors do not visually imply invalid actions. (b) Q-value landscape showing a smooth high-value basin near safe goal-reaching trajectories. (c) Untuned artificial potential field: the left panel shows a local-minimum trap, while the right panel shows the trained stage-4 SAC policy reaching the goal from the same start. (d) Representative trajectory traces. Together these plots suggest that the policy learned a potential-well-like structure without inheriting the classical local-minimum failure of a naive potential field.

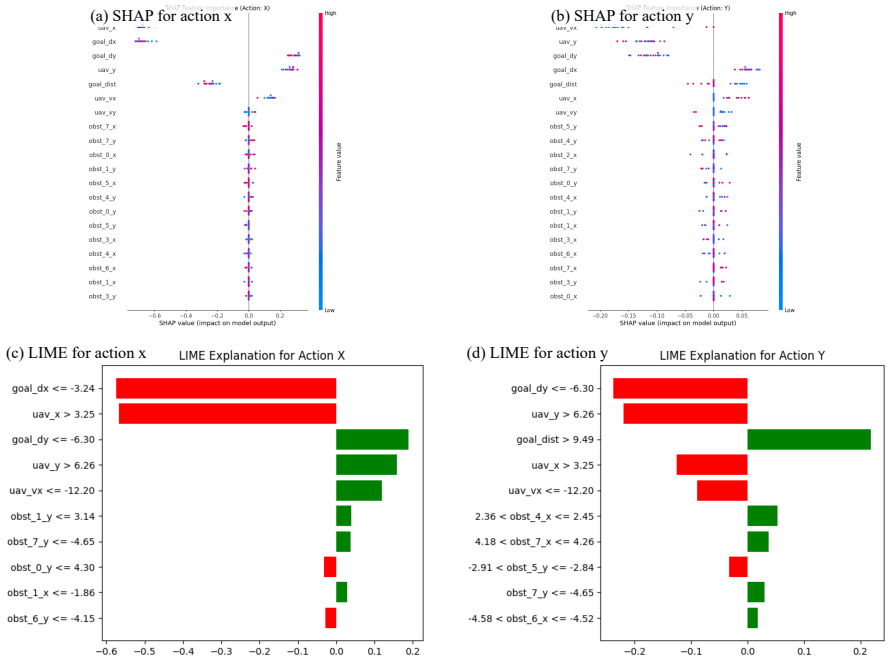

Fig. 4. Post hoc explanations for the main SAC policy. SHAP gives global feature-importance summaries for both action dimensions, while LIME highlights local directional explanations around a representative state. Goal displacement and velocity dominate globally, but local obstacle-aware corrections remain necessary for faithful closed-loop behavior.

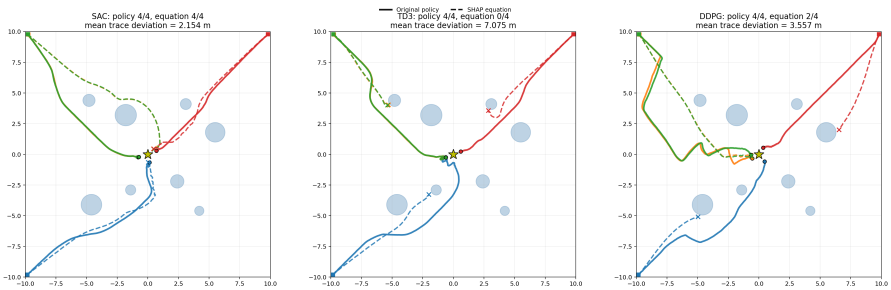

Fig. 5. Stage-4 trajectory overlays for the original policies and their final rollout-selected SHAP equations. SAC remains the strongest case: the distilled analytical controller preserves all four held-out successes. DDPG is partially faithful, while TD3 diverges in closed loop despite successful original rollouts.